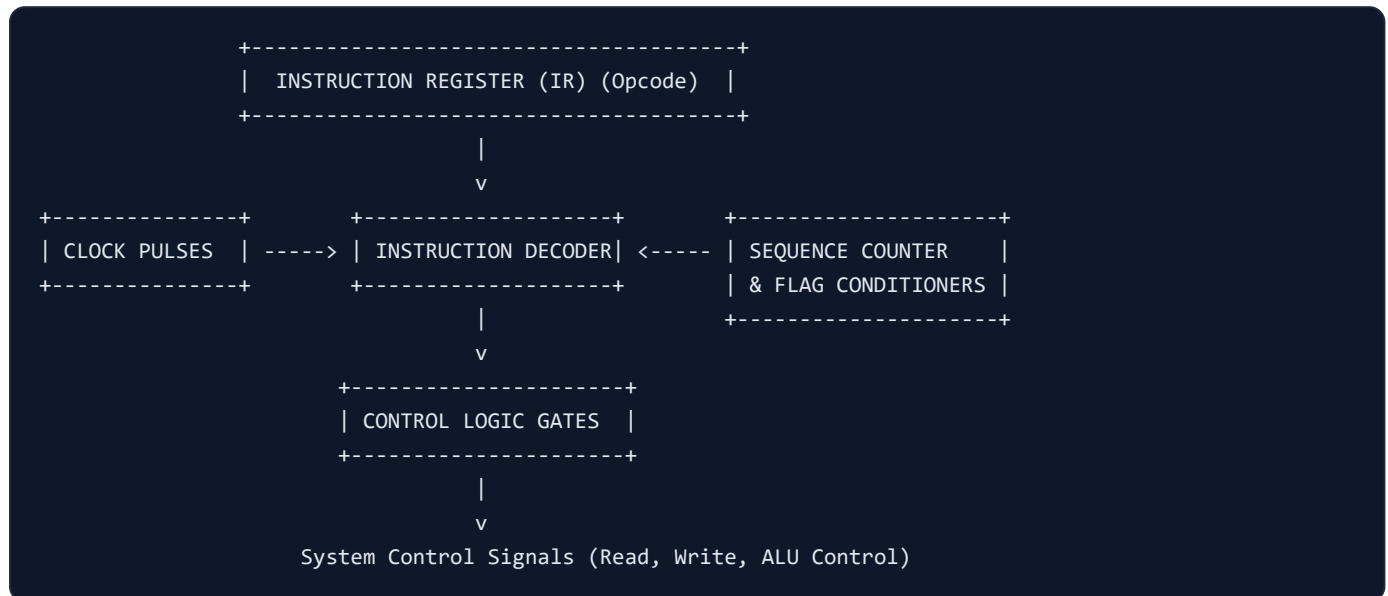


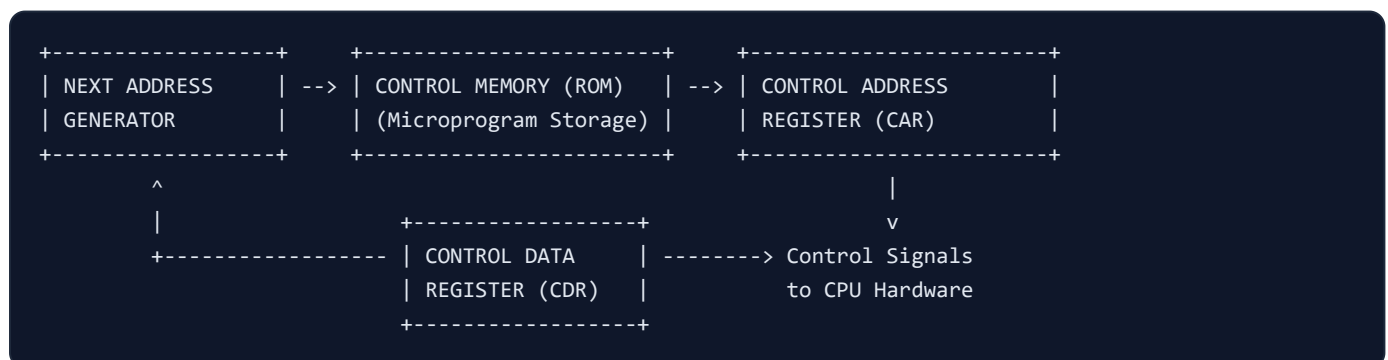
5-Mark Questions & Answers — MPCA Unit 2: Processor Organization & Architecture

Q1. Explain hardwired control unit design with a block diagram.



1. **Opcode Decoder:** Decodes machine instruction opcode into active signal lines.
2. **Sequence Counter:** Generates timing signals (**T₀, T₁, T₂, ...**) for sequential step execution.
3. **Control Logic Combinational Circuit:** Logic gates synthesize control signals (**C_{read}, C_{write}, ALU_{add}**) dynamically based on opcode and condition flags.
4. **Pros & Cons:** Ultra-high speed execution, but extremely difficult to modify or debug once fabricated.

Q2. Explain microprogrammed control unit with microinstruction sequencing and execution.



1. **Control Memory (ROM):** Holds sequences of microinstructions (microprogram) for every machine instruction opcode.
2. **Control Address Register (CAR):** Contains the memory address of the next microinstruction to fetch.
3. **Control Data Register (CDR) / Control Buffer:** Holds the currently fetched microinstruction containing control signals and next-address fields.

4. Sequencing & Execution:

- **Sequencer Logic:** Determines whether to increment **CAR** or branch to a target microinstruction address based on condition flags.

Q3. Explain instruction pipelining with a diagram and discuss speedup achieved.

```
Clock Cycle : C1    C2    C3    C4    C5    C6    C7
-----
Instr 1      : [ IF ][ ID ][ EX ][ MEM ][ WB ]
Instr 2      :      [ IF ][ ID ][ EX ][ MEM ][ WB ]
Instr 3      :          [ IF ][ ID ][ EX ][ MEM ][ WB ]
```

Speedup Calculation

For a **k-stage pipeline** executing **n instructions**:

- **Non-Pipelined Time:** $T_{\text{non_pipe}} = n \times k \times \tau$

- **Pipelined Time:** $T_{\text{pipe}} = (k + n - 1) \times \tau$

- **Speedup Ratio (S):**

$$\text{Speedup (S)} = T_{\text{non_pipe}} / T_{\text{pipe}} = (n \times k \times \tau) / [(k + n - 1) \times \tau]$$

- For a large number of instructions ($n \rightarrow \infty$):

$$\text{Maximum Speedup (S}_{\text{max}}) = k \text{ (Equal to number of pipeline stages)}$$

Q4. Explain different types of pipeline hazards and techniques to overcome them.

1. Structural Hazards (Hardware Conflicts):

- *Cause:* Multiple stages attempting simultaneous access to a shared resource.
- *Solution:* Duplicate resources (e.g., separate Instruction Cache and Data Cache).

2. Data Hazards (Data Dependencies):

- *Types:* Read-After-Write (RAW), Write-After-Read (WAR), Write-After-Write (WAW).
- *Solution:* **Data Forwarding / Bypassing** (routing ALU output directly back to ALU input), or inserting **Pipeline Stalls (NOPs/Bubbles)**.

3. Control Hazards (Branch Dependencies):

- *Cause:* Conditional branch decisions delaying instruction fetch stream.
- *Solution:* **Branch Prediction** (Static/Dynamic), **Branch Target Buffers (BTB)**, or **Delayed Branching**.

Q5. Describe Flynn's classification of parallel processing architectures in detail with diagrams.

```
+-----+-----+
|          SINGLE DATA          |          MULTIPLE DATA          |
+-----+-----+
|SINGLE|  SISD (Single Inst, Single Data) | SIMD (Single Inst, Multiple Data)|
|INST |  [CU] -> [PE] -> [Memory]       | [CU] -> [PE1, PE2, PE3]       |
+-----+-----+
|MULTI|  MISD (Multi Inst, Single Data) | MIMD (Multi Inst, Multiple Data)|
|INST |  [CU1, CU2] -> [Single Data]     | [CU1, CU2] -> [PE1, PE2]     |
+-----+-----+
```

1. **SISD**: Standard uniprocessor system.
 2. **SIMD**: Vector processors, GPUs, SIMD extensions (AVX-512) for parallel array calculations.
 3. **MISD**: Specialized fault-tolerant computers (redundant processing on identical data).
 4. **MIMD**: Multi-core processors, distributed computing clusters, supercomputers.
-

Q6. Explain the basic instruction cycle along with interrupt processing in detail.

1. **Fetch Phase**: CPU fetches opcode byte from memory location specified by **PC** into **IR**. Program Counter is incremented ($PC = PC + 1$).
2. **Decode Phase**: Control unit decodes opcode to determine operation and operand locations.
3. **Operand Fetch Phase**: Effective address is calculated and operands loaded into data registers.
4. **Execute Phase**: ALU performs arithmetic or logical operation.
5. **Interrupt Processing Phase**:
 - At the end of every instruction cycle, the CPU polls the Interrupt request line.
 - If an interrupt is active and enabled:
 1. Current **PC** and **FLAGS** are pushed onto the Stack.
 2. Interrupt vector address is loaded into **PC**.
 3. Execution jumps to the Interrupt Service Routine (ISR).
 4. **IRET** pops saved state back to resume original program execution.